

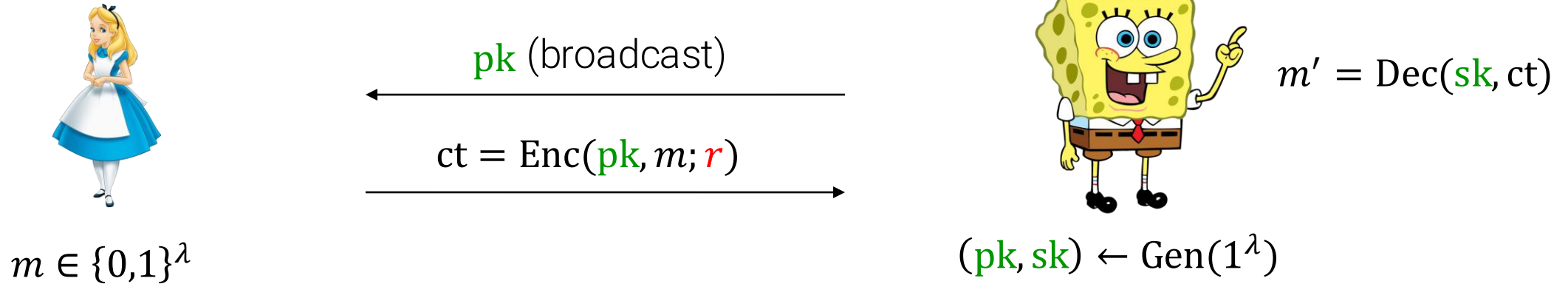
COS 433: Cryptography

Announcements:

- Problem Set 3 due 10/29
- Lecture notes on computational number theory (by Dana Angluin) on the course website

Public-Key Encryption

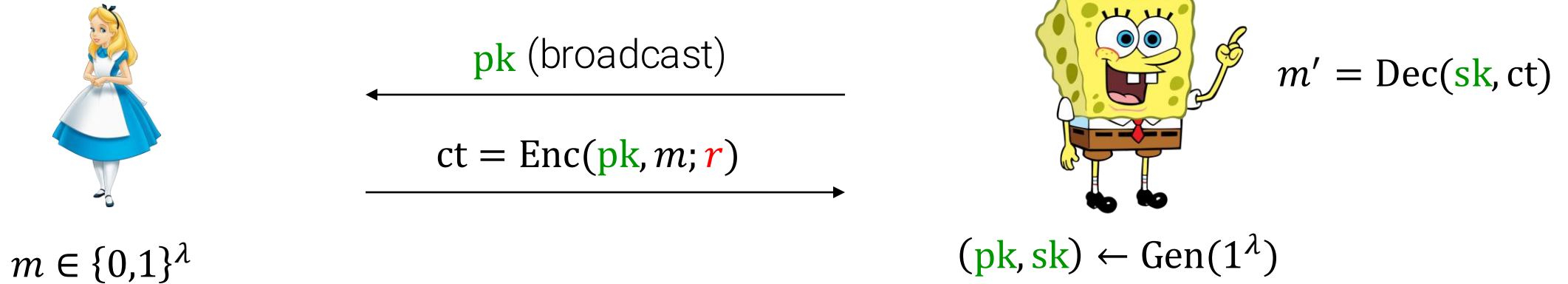
Public-Key Encryption



Definition: A public-key encryption scheme consists of algorithms $(\text{Gen}, \text{Enc}, \text{Dec})$. It must satisfy the following properties:

- **Efficiency:** $\text{Gen}, \text{Enc}, \text{Dec}$ run in polynomial time
- **Correctness:** $\text{Dec}(\text{sk}, \text{Enc}(\text{pk}, m)) = m$ for all $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$.
- **Computational security:** For all m_0, m_1 , $(\text{pk}, \text{Enc}(\text{pk}, m_0)) \approx_c (\text{pk}, \text{Enc}(\text{pk}, m_1))$

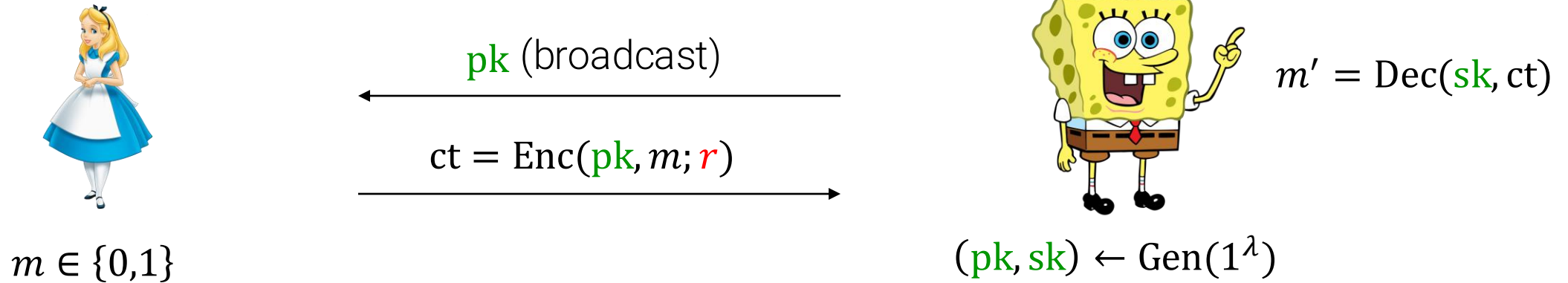
Public-Key Encryption



Theorem (Goldwasser-Micali): under the QR assumption, public-key encryption exists!

PKE has many nice properties!

Public-Key Encryption (one-bit messages)



Definition: A public-key encryption scheme consists of algorithms $(\text{Gen}, \text{Enc}, \text{Dec})$. It must satisfy the following properties:

- **Efficiency:** $\text{Gen}, \text{Enc}, \text{Dec}$ run in polynomial time
- **Correctness:** $\text{Dec}(sk, \text{Enc}(pk, m)) = m$ for all $(pk, sk) \leftarrow \text{Gen}(1^\lambda)$.
- **Computational security:** $(pk, \text{Enc}(pk, 0)) \approx_c (pk, \text{Enc}(pk, 1))$

CPA Security for PKE

Challenger



pk

$b \leftarrow \{0,1\}$

pk

$m_0^{(\lambda)}, m_1^{(\lambda)}$

$\text{Enc}(\text{pk}, m_b; r)$

Adversary



b'

Win if $b' = b$

Definition: A public-key encryption scheme with message length $n = n(\lambda)$ satisfies **IND-CPA security** if

For all polynomial-time



$$\Pr[\text{Win}] \leq \frac{1}{2} + \text{negl}(\lambda)$$

CPA Security for PKE (equivalent)

Challenger



pk

$b \leftarrow \{0,1\}$

pk

$m_0^{(\lambda)}, m_1^{(\lambda)}$

$\text{Enc}(\text{pk}, m_b; r)$

Adversary



b'

Win if $b' = b$

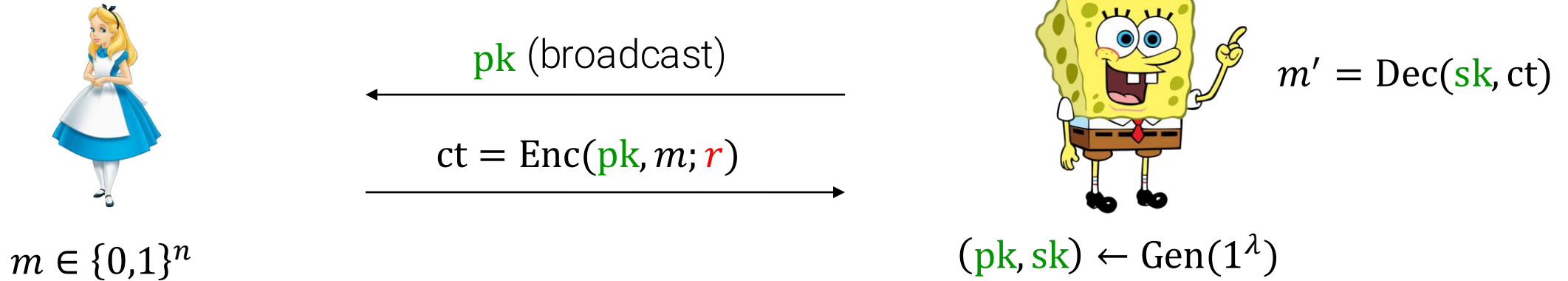
Definition: A public-key encryption scheme with message length $n = n(\lambda)$ satisfies **IND-CPA security** if

For all polynomial-time



$$\Pr[\text{Win}] \leq \frac{1}{2} + \text{negl}(\lambda)$$

Public-Key Encryption



Lemma: Suppose that $(\text{Gen}, \text{Enc}, \text{Dec})$ is a public-key encryption scheme with messages of length $n = 1$. Then, there exists a PKE with message length $n(\lambda)$ for all polynomial functions $n(\lambda)$, satisfying **CPA-security**.

Proof: encrypt bit-by-bit. Security holds by a hybrid argument.

Next: RSA Encryption and Diffie-Hellman

RSA Computational Problem

How hard is it to solve equations **in one variable** over $\mathbb{Z}_N$?

As hard as factoring: Given a , find x such that $x^2 \equiv a \pmod{N}$

QR (believed hard): Given a , decide if $x^2 \equiv a \pmod{N}$ has a solution

What about equations of higher degree?

RSA Computational Problem

How hard is it to solve equations **in one variable** over $\mathbb{Z}_N$?

Cube root finding (believed hard): Given a , find x such that $x^3 \equiv a \pmod{N}$

Candidate OWF:

$$f(p, q, x) = (pq, x^3 \bmod pq)$$

Cube root mod N assumption

Assumption:

For all poly-time A ,

$$\Pr_{\substack{p,q \leftarrow \{0,1\}^\lambda \\ p,q \text{ prime}, N=pq \\ x \leftarrow \mathbb{Z}_N^\times}} [A(N, x^3 \bmod N) = y: y^3 \equiv x^3 \bmod N] = \text{negl}(\lambda)$$

RSA Computational Problem

Cube root finding (believed hard): Given a , find x such that $x^3 \equiv a \pmod{N}$

How does this compare to square root finding? Depends on p, q

Tool: Euler's theorem. For all $a \in \mathbb{Z}_N^\times$ ($\gcd(a, N) = 1$),

$$a^{\phi(N)} \equiv 1 \pmod{N}$$

where $\phi(N) = (p - 1)(q - 1) = |\mathbb{Z}_N^\times|$.

RSA Computational Problem

Cube root finding (believed hard): Given a , find x such that $x^3 \equiv a \pmod{N}$

How does this compare to square root finding? Depends on p, q

Tool: Euler's theorem. For all $a \in \mathbb{Z}_N^\times$ ($\gcd(a, N) = 1$),

$$a^{\phi(N)} \equiv 1 \pmod{N}$$

Case 1: if 3 divides $\phi(N)$, then $x \mapsto x^3$ is many-to-one, and cube root-finding is **as hard as factoring**! (But it's **not too different from square root finding**.)

RSA Computational Problem

Cube root finding (believed hard): Given a , find x such that $x^3 \equiv a \pmod{N}$

How does this compare to square root finding? Depends on p, q

Tool: Euler's theorem. For all $a \in \mathbb{Z}_N^\times$ ($\gcd(a, N) = 1$),

$$a^{\phi(N)} \equiv 1 \pmod{N}$$

Case 2: if 3 **does not divide** $\phi(N)$, then $x \mapsto x^3$ is a permutation on $\mathbb{Z}_N^\times$!

Reason: $\exists d: 3d \equiv 1 \pmod{\phi(N)}$. $y \mapsto y^d$ is an inverse map!

RSA Computational Problem

Cube root finding (believed hard): Given a , find x such that $x^3 \equiv a \pmod{N}$

How does this compare to square root finding? Depends on p, q

Tool: Euler's theorem. For all $a \in \mathbb{Z}_N^\times$ ($\gcd(a, N) = 1$),

$$a^{\phi(N)} \equiv 1 \pmod{N}$$

Case 2: if 3 **does not divide** $\phi(N)$, then $x \mapsto x^3$ is a permutation on $\mathbb{Z}_N^\times$!

But: no longer know that cube root finding is as hard as factoring.

RSA Computational Problem

Candidate OWF:

$$f_N(x) = x^3 \bmod N$$

When 3 **does not divide** $(p - 1)(q - 1)$, this is a **Trapdoor Permutation**

- One-way permutation (defined on $\mathbb{Z}_N^\times$)
- Given **trapdoor** d , permutation becomes easy to invert!
- "RSA with exponent 3"

RSA Computational Problem

Another candidate trapdoor permutation:

$$f_{N,e}(x) = x^e \bmod N$$

where $\gcd(e, \phi(N)) = 1$. Typical: sample $e \leftarrow \mathbb{Z}_{\phi(N)}^\times$.

- “RSA trapdoor permutation”
- Given p, q , can find trapdoor d : $e \cdot d \equiv 1 \pmod{\phi(N)}$.
- $(x^e)^d \equiv x^{ed} \equiv x^1 \equiv x \bmod N$

RSA Assumption

Assumption:

For all poly-time A ,

$$\Pr_{\substack{p,q \leftarrow \{0,1\}^\lambda \\ p,q \text{ prime}, N=pq \\ e \leftarrow \mathbb{Z}_{\phi(N)}^\times \\ x \leftarrow \mathbb{Z}_N^\times}} [A(N, e, x^e \bmod N) = x] = \text{negl}(\lambda)$$

Open: prove factoring assumption $\Leftrightarrow$ RSA assumption

RSA Assumption

Assumption:

For all poly-time A ,

$$\Pr_{\substack{p,q \leftarrow \{0,1\}^\lambda \\ p,q \text{ prime}, N=pq \\ e \leftarrow \mathbb{Z}_{\phi(N)}^\times \\ x \leftarrow \mathbb{Z}_N^\times}} [A(N, e, x^e \bmod N) = x] = \text{negl}(\lambda)$$

Exercise: finding inverse $d \iff$ factoring.

“Textbook RSA Encryption”

Heuristic PKE scheme

$$\text{Gen}(1^\lambda) \rightarrow (\mathbf{pk} = (N, e), \mathbf{sk} = d)$$

$$\text{Enc}(\mathbf{pk}, m) = m^e \bmod N.$$

$$\text{Dec}(\mathbf{sk}, \text{ct}) = \text{ct}^d \bmod N.$$

- Correct!
- Hard to recover random m given ct
- **Deterministic**, cannot satisfy any of our security definitions

“Textbook RSA Encryption”

Heuristic PKE scheme

$$\text{Gen}(1^\lambda) \rightarrow (\mathbf{pk} = (N, e), \mathbf{sk} = d)$$

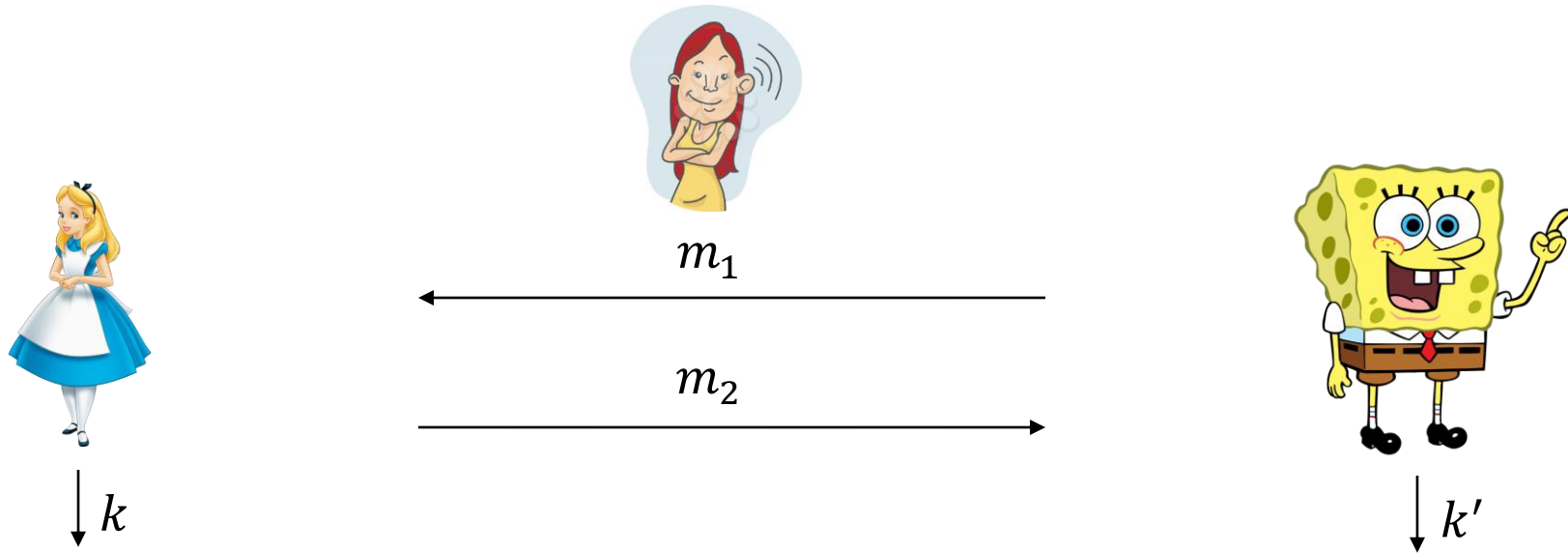
$$\text{Enc}(\mathbf{pk}, m) = m^e \bmod N.$$

$$\text{Dec}(\mathbf{sk}, \text{ct}) = \text{ct}^d \bmod N.$$

- Can be turned into a secure PKE in several ways (see HW3 for one way)
- Efficient CCA-secure variant of RSA used in practice (RSA-OAEP)

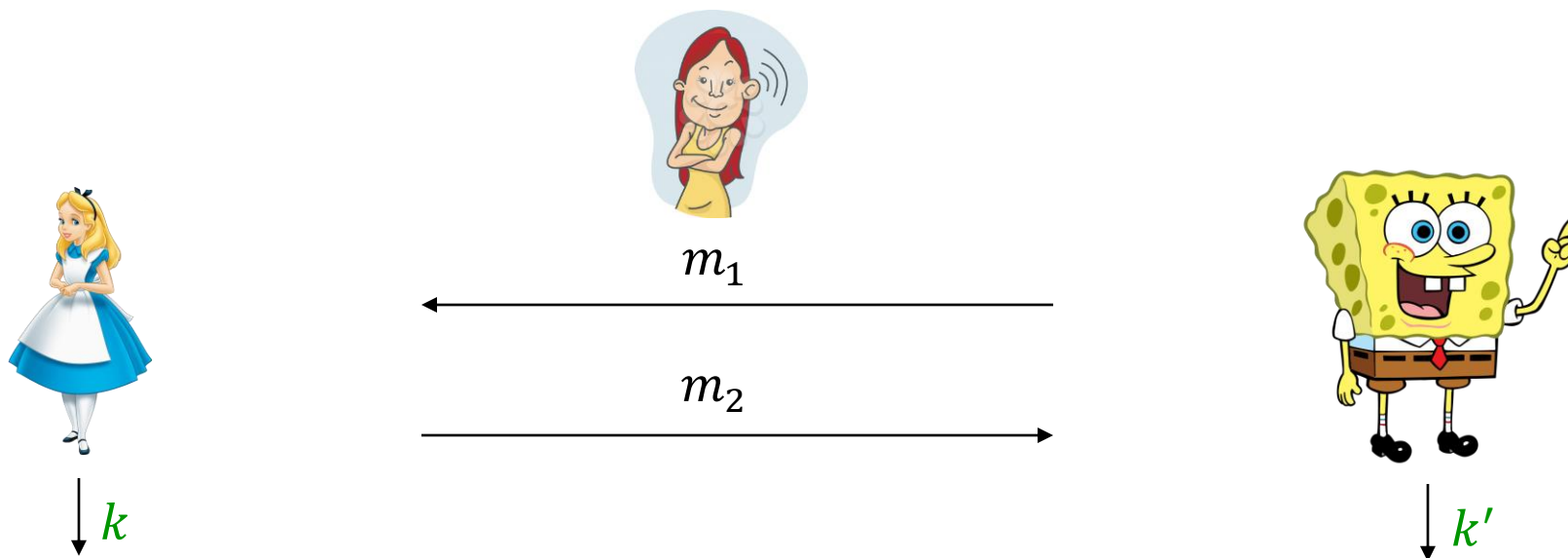
Diffie-Hellman “Key Exchange”

Key Agreement



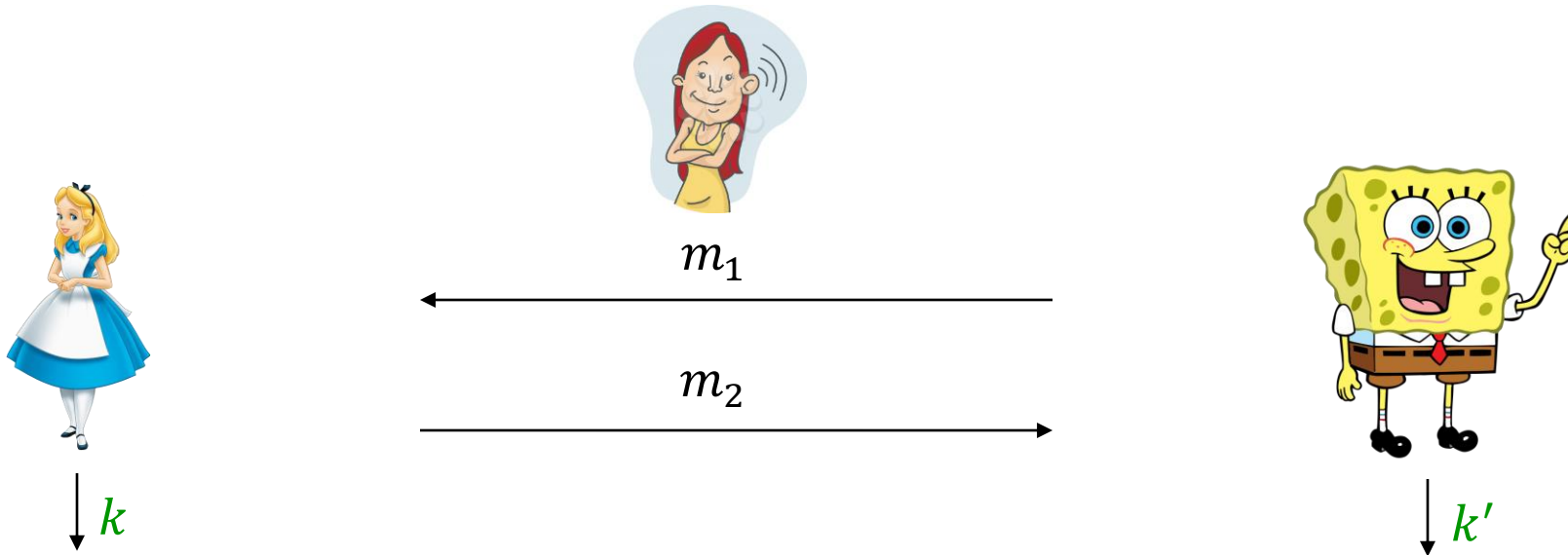
- Goal: share a secret key k that looks uniformly random to any 
- Alice and Bob do not want to meet in person
- They run a **protocol** (Bob sends a message, then Alice sends a message)

Key Agreement



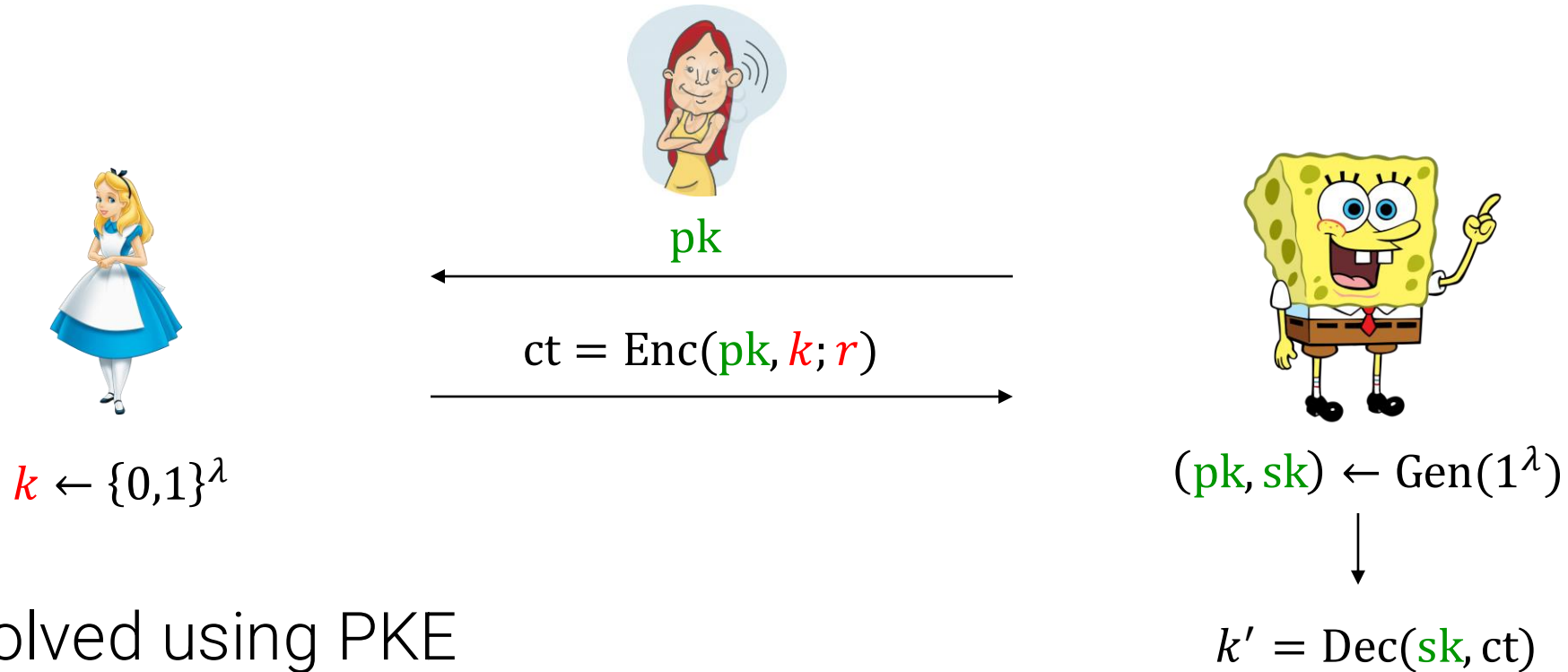
- Correctness: if Alice and Bob behave honestly, then $k = k'$
- Weak security: hard for  to find k given (m_1, m_2) .
- Strong security: $(m_1, m_2, k) \approx_c (m_1, m_2, u)$ (uniformly random key u)

Key Agreement



Can be solved using PKE

Key Agreement



Can be solved using PKE

- Strong security follows from PKE security
- Protocol establishes a **session key** which can be used to encrypt new messages using SKE (e.g. AES)

To be continued